

of the business of the largest computer company in the world. Register-memory architectures were the most popular in the 1960s and the first half of the 1970s.

Digital Equipment Corporation's VAX architecture took memory operands one step further in 1977. It allowed an instruction to use any combination of registers and memory operands. A style of architecture in which all operands can be in memory is called *memory-memory*. (In truth the VAX instruction set, like almost all other instruction sets since the IBM 360, is a hybrid, since it also has general-purpose registers.)

The Intel x86 has many versions of a 64-bit add to specify whether an operand is in memory or is in a register. In addition, the memory operand can be accessed with more than seven addressing modes. This combination of address modes and register-memory operands means that there are dozens of variants of an x86 add instruction. Clearly, this variability makes x86 implementations more challenging.

Compact Code and Stack Architectures

When memory is scarce, it is also important to keep programs small, so architectures like the Intel x86, IBM 360, and VAX had variable-length instructions, both to match the varying operand specifications and to minimize code size. Intel x86 instructions are from 1 to 15 bytes long; IBM 360 instructions are 2, 4, or 6 bytes long; and VAX instruction lengths are anywhere from 1 to 54 bytes.

One place where code size is still important is embedded applications. In recognition of this need, both ARM and MIPS made versions of their instructions sets that offer both 16-bit instruction formats and 32-bit instruction formats: Thumb and Thumb-2 for ARM, and MIPS-16. Despite being limited to just two sizes, Thumb, Thumb-2, and MIPS-16 programs are about 25% to 30% smaller, which makes their code sizes smaller than those of the 80x86. Smaller code sizes have the added benefit of improving instruction cache hit rates (see [Chapter 5](#)).

In the 1960s, a few companies followed a radical approach to instruction sets. In the belief that it was too hard for compilers to utilize registers effectively, these companies abandoned registers altogether! Instruction sets were based on a *stack model* of execution, like that found in the older Hewlett-Packard handheld calculators. Operands are pushed on the stack from memory or popped off the stack into memory. Operations take their operands from the stack and then place the result back onto the stack. In addition to simplifying compilers by eliminating register allocation, stack architectures lent themselves to compact instruction encoding, thereby removing memory size as an excuse not to program in high-level languages.

Memory space was perceived to be precious again for Java, both because memory space is limited to keep costs low in embedded applications and because programs may be downloaded over the Internet or phone lines as Java applets, and smaller programs take less time to transmit. Hence, compact instruction encoding was desirable for Java bytecodes.

High-Level-Language Computer Architectures

In the 1960s, systems software was rarely written in high-level languages. For example, virtually every commercial operating system before UNIX was programmed in assembly language, and more recently even OS/2 was originally programmed at that same low level. Some people blamed the code density of the instruction sets, rather than the programming languages and the compiler technology.

Hence, an architecture design philosophy called *high-level-language computer architecture* was advocated, with the goal of making the hardware more like the programming languages. More efficient programming languages and compilers, plus expanding memory, doomed this movement to a historical footnote. The Burroughs B5000 was the commercial fountainhead of this philosophy, but today there is no significant commercial descendant of this 1960s radical.

Reduced Instruction Set Computer Architectures

This language-oriented design philosophy was replaced in the 1980s by RISC (*reduced instruction set computer*). Improvements in programming languages, compiler technology, and memory cost meant that less programming was being done at the assembly level, so instruction sets could be measured by how well compilers used them, in contrast to how skillfully assembly language programmers used them.

Virtually all new instruction sets since 1982 have followed this RISC philosophy of fixed instruction lengths, load-store instruction sets, limited addressing modes, and limited operations. ARMv7, ARMv8 Hitachi SH, IBM PowerPC, MIPS, and Sun SPARC are all examples of RISC architectures.

A Brief History of the ARMv7

ARM started as the processor for the Acorn computer, hence its original name of Acorn RISC Machine. The Berkeley RISC papers influenced its architecture.

One of the most important early applications was emulation of the AM 6502, a 16-bit microprocessor. This emulation was to provide most of the software for the Acorn computer. As the 6502 had a variable-length instruction set that was a multiple of bytes, 6502 emulation helps explain the emphasis on shifting and masking in the ARMv7 instruction set.

Its popularity as a low-power embedded computer began with its selection as the processor for the ill-fated Apple Newton personal digital assistant. Although the Newton was not as popular as Apple hoped, Apple's blessing gave visibility to the earlier ARM instruction sets, and they subsequently caught on in several markets, including cell phones. Unlike the Newton experience, the extraordinary success of cell phones explains why 12 billion ARM processors were shipped in 2014.

One of the major events in ARM's history is the 64-bit address extension called version 8. ARM took the opportunity to redesign the instruction set to make it look much more like MIPS than like earlier ARM versions.

A Brief History of the x86

The ancestors of the x86 were the first microprocessors, produced starting in 1972. The Intel 4004 and 8008 were extremely simple 4-bit and 8-bit accumulator-style architectures. [Morse et al. \[1980\]](#) describe the evolution of the 8086 from the 8080 in the late 1970s as an attempt to provide a 16-bit architecture with better throughput. At that time, almost all programming for microprocessors was done in assembly language—both memory and compilers were in short supply. Intel wanted to keep its base of 8080 users, so the 8086 was designed to be “compatible” with the 8080. The 8086 was *never* object-code compatible with the 8080, but the architectures were close enough that translation of assembly language programs could be done automatically.

In early 1980, IBM selected a version of the 8086 with an 8-bit external bus, called the 8088, for use in the IBM PC. They chose the 8-bit version to reduce the cost of the architecture. This choice, together with the tremendous success of the IBM PC, has made the 8086 architecture ubiquitous. The success of the IBM PC was due in part because IBM opened the architecture of the PC and enabled the PC-clone industry to flourish. As discussed in [Section 2.18](#), the 80286, 80386, 80486, Pentium, Pentium Pro, Pentium II, Pentium III, Pentium 4, and AMD64 have extended the architecture and provided a series of performance enhancements.

Although the 68000 was chosen for the Macintosh, the Mac was never as pervasive as the PC, partly because Apple did not allow Mac clones based on the 68000, and the 68000 did not acquire the same software following that which the 8086 enjoys. The Motorola 68000 may have been more significant *technically* than the 8086, but the impact of IBM’s selection and open architecture strategy dominated the technical advantages of the 68000 in the market.

Some argue that the inelegance of the x86 instruction set is unavoidable, the price that must be paid for rampant success by any architecture. We reject that notion. Obviously, no successful architecture can jettison features that were added in previous implementations, and over time, some features may be seen as undesirable. The awkwardness of the x86 begins at its core with the 8086 instruction set and was exacerbated by the architecturally inconsistent expansions found in the 8087, 80286, 80386, MMX, SSE, SSE2, SSE3, SSE4, AMD64 (EM64T), and AVX.

A counterexample is the IBM 360/370 architecture, which is much older than the x86. It dominated the mainframe market just as the x86 dominated the PC market. Due undoubtedly to a better base and more compatible enhancements, this instruction set makes much more sense than the x86 50 years after its first implementation.

Extending the x86 to 64-bit addressing means the architecture may last for several more decades. Instruction set anthropologists of the future will peel off layer after layer from such architectures until they uncover artifacts from the first microprocessor. Given such a find, how will they judge today’s computer architecture?

A Brief History of Programming Languages

In 1954, John Backus led a team at IBM to create a more natural notation for scientific programming. The goal of Fortran, for “FORmula TRANslator,” was to reduce the time to develop programs. Fortran included many ideas found in programming languages today, including assignment statements, expressions, typed variables, loops, and arrays. The development of the language and the compiler went hand in hand. This language became a standard that has evolved over time to improve programmer productivity and program portability. The evolutionary steps are Fortran I, II, IV, 77, and 90.

Fortran was developed for IBM’s second commercial computer, the 704, which was also the cradle of another important programming language: Lisp. John McCarthy invented the “LIST Processing” language in 1958. Its mantra is that programming can be considered as manipulating lists, so the language contains operations to follow links and to compose new lists from old ones. This list notation is used for the code as well as the data, so modifying or composing Lisp programs is common. The big contribution was dynamic data structures and, hence, pointers. Given that its inventor was a pioneer in artificial intelligence, Lisp became popular in the AI community. Lisp has no type declarations, and Lisp traditionally reclaims storage automatically via built-in garbage collection. Lisp was originally interpreted, although compilers were later developed for it.

Fortran inspired the international community to invent a programming language that was more natural to express algorithms than Fortran, with less emphasis on coding. This language became Algol, for “ALGOrithmic Language.” Like Fortran, it included type declarations, but it added recursive procedure calls, nested *if-then-else* statements, *while* loops, *begin-end* statements to structure code, and call-by-name. Algol-60 became the classic language for academics to teach programming in the 1960s.

Although engineers, AI researchers, and computer scientists had their own programming languages, the same could not be said for business data processing. Cobol, for “COmmon Business-Oriented Language,” was developed as a standard for this purpose contemporary with Algol-60. Cobol was created to be easy to read, so it follows English vocabulary and punctuation. It added records to programming languages, and separated description of data from description of code.

Niklaus Wirth was a member of the Algol-68 committee, which was supposed to update Algol-60. He was bothered by the complexity of the result, and so he wrote a minority report to show that a programming language could combine the algorithmic power of Algol-60 with the record structure from Cobol and be simple to understand, easy to implement, yet still powerful. This minority report became Pascal. It was first implemented with an interpreter and a set of Pascal bytecodes. The ease of implementation led to its being widely deployed, much more than Algol-68, and it soon replaced Algol-60 as the most popular language for academics to teach programming.

In the same period, Dennis Ritchie invented the C programming language to use in building UNIX. Its inventors say it is not a “very high level” programming language or a big one, and it is not aimed at a particular application. Given its birthplace, it was very good at systems programming, and the UNIX operating system and C compiler were written in C. UNIX’s popularity helped spur C’s popularity.

The concept of object orientation is first captured in Simula-67, a simulation language successor to Algol-60. Invented by Ole-Johan Dahl and Kristen Nygaard at the University of Oslo in 1967, it introduced objects, classes, and inheritance.

Object orientation proved to be a powerful idea. It led Alan Kay and others at Xerox Palo Alto Research Center to invent Smalltalk in the 1970s. Smalltalk-80 married the typeless variables and garbage collection from Lisp and the object orientation of Simula-67. It relied on interpretation that was defined by a Smalltalk virtual machine with a Smalltalk bytecode instruction set. Kay and his colleagues argued that processors were getting faster, and that we must eventually be willing to sacrifice some performance to improve program development. Another example was CLU, which demonstrated that an object-oriented language could be defined that allowed compile-time type checking. Simula-67 also inspired Bjarne Stroustrup of Bell Labs to develop an object-oriented version of C called C++ in the 1980s. C++ became widely used in industry.

Dissatisfied with C++, a group at Sun led by James Gosling invented Oak in the early 1990s. It was invented as an object-oriented C dialect for embedded devices as part of a major Sun project. To make it portable, it was interpreted and had its own virtual machine and bytecode instruction set. Since it was a new language, it had a more elegant object-oriented design than C++ and was much easier to learn and compile than Smalltalk-80. Since Sun’s embedded project failed, we might never have heard of it had someone not made the connection between Oak and programmable browsers for the World Wide Web. It was rechristened Java, and in 1995, Netscape announced that it would be shipping with its browser. It soon became extraordinarily popular. Java had the rare distinction of becoming the standard language for new business data processing applications *and* the favored language for academics to teach programming. Java and languages like it encourage reuse of code, and hence programmers make heavy use of libraries, whereas in the past they were more likely to write everything from scratch.

A Brief History of Compilers

Backus and his group were very concerned that Fortran would be unsuccessful if skeptics found examples where the Fortran version ran at half the speed of the equivalent assembly language program. Their success with one of the first compilers created a beachhead that many others followed.

Early compilers were ad hoc programs that performed the steps described in [Section 2.15](#) online. These ad hoc approaches were replaced with a solid theoretical foundation for each of these steps. Each time the theory was established, a tool was built based on that theory that automated the creation of that step.

The theoretical roots underlying scanning and parsing derive from automata theory, and the relationship between languages and automata was known early. The scanning task corresponds to recognition of a language accepted by a finite-state automata, and parsing corresponds to recognition of a language by a push-down automata (basically an automata with a stack). Languages are described by grammars, which are a set of rules that tell how any legal program can be generated.

The scanning pass of a compiler was well understood early, but parsing is harder. The earliest parsers use precedence techniques, which derived from the structure of arithmetic statements, and were then generalized. The great breakthrough in modern parsing was made by Donald Knuth in the invention of LR-parsing, which codified the two key steps in the parsing technique, pushing a token on the stack or reducing a set of tokens on the stack using a grammar rule. The strong theory formulation for scanning and parsing led to the development of automated tools for compiler constructions, such as `lex` and `yacc`, the tools developed as part of UNIX.

Optimizations occurred in many compilers, and it is harder to determine the first examples in most cases. However, Victor Vyssotsky did the first papers on data flow analysis in 1963, and William McKeeman is generally credited with the first peephole optimizer in 1965. The group at IBM, including John Cocke and Fran Allan, developed many of the early optimization concepts, as well as defining and extending the concepts of flow analysis. Important contributions were also made by Al Aho and Jeff Ullman.

One of the biggest challenges for optimization was register allocation. It was so difficult that some architects used stack architectures just to avoid the problem. The breakthrough came when researchers working on compilers for the 801, an early RISC architecture, recognized that coloring a graph with a minimum number of colors was equivalent to allocating a fixed number of registers to the unlimited number of virtual registers used in intermediate forms.

Compilers also played an important role in the open-source movement. Richard Stallman's self-appointed mission was to make a public domain version of UNIX. He built the GNU C Compiler (`gcc`) as an open-source compiler in 1987. It soon was ported to many architectures, and is used in many systems today.

Further Reading

Bayko, J. [1996]. "Great microprocessors of the past and present," search for it on the <http://www.cpushack.com/CPU/cpu.html>.

A personal view of the history of both representative and unusual microprocessors, from the Intel 4004 to the Patriot Scientific ShBoom!

Kane, G. and J. Heinrich [1992]. *MIPS RISC Architecture*, Prentice Hall, Englewood Cliffs, NJ.

This book describes the MIPS architecture in greater detail than Appendix A.

Levy, H. and R. Eckhouse [1989]. *Computer Programming and Architecture, The VAX*, Digital Press, Boston.

This book concentrates on the VAX, but also includes descriptions of the Intel 8086, IBM 360, and CDC 6600.

Morse, S., B. Ravenal, S. Mazor, and W. Pohlman [1980]. “Intel microprocessors—8080 to 8086”, *Computer* 13 10 (October).

The architecture history of the Intel from the 4004 to the 8086, according to the people who participated in the designs.

Wakerly, J. [1989]. *Microcomputer Architecture and Programming*, Wiley, New York.

The Motorola 6800 is the main focus of the book, but it covers the Intel 8086, Motorola 6809, TI 9900, and Zilog Z8000.

architectures, and a brief history of ARMv7 and the x86. We also review the controversial subjects of high-level-language computer architectures and reduced instruction set computer architectures. The history of programming languages includes Fortran, Lisp, Algol, C, Cobol, Pascal, Simula, Smalltalk, C++, and Java, and the history of compilers includes the key milestones and the pioneers who achieved them. The rest of  [Section 2.22](#) is found online.

2.23

Exercises

(Readers may wish to download and install a free Community Edition of the ARM DS-5 professional software suite which contains an ARMv8-A (64-bit) architecture simulator before attempting these exercises. More details are available on the companion Web site for this book.)

The main difference between LEGv8 and ARMv8, is that LEGv8 is a subset of ARMv8. Thus, compilers may generate ARMv8 instructions that are not found in LEGv8. That is one reason we list the rest of the ARMv8 instructions in Sections 2.19, 3.8, and 5.14. If you were to look at the output of a compiler, the most obvious difference would be the likely use of scaled loads and stores (LDR and STR) instead of the unscaled versions that we feature in LEGv8 (LDUR and STUR).

We also made some changes to ARMv8 for pedagogic reasons in LEGv8, which we noted in the *Elaboration* sections in the text, that will not work with the ARMv8 assembler and simulators. Here are the four differences:

- Immediate versions are not separate assembly language instructions in ARMv8. While LEGv8 has `ADDI`, `SUBI`, `ADDIS`, `SUBIS`, `ANDI`, `ORRI`, and `EORI`, ARMv8 does not. You simply write the non-immediate version and use an immediate operand. For example, the ARMv8 assembler accepts

```
ADD X0, X1, #4
```

and produces the immediate version of the opcode for `ADD`, which LEGv8 associates with `ADDI`.

- Similarly, single- and double-precision floating-point instructions do not have separate assembly language instructions. Thus, ARMv8 does not have `FADDs`, `FADDd`, `FSUBs`, `FSUBd`, `FMULs`, `FMULD`, `FDIVs`, `FDIVd`, `FCMPS`, `FCMPD`, `LDURs`, `LDURd`, `STURs`, and `STURd`, as does LEGv8. Instead, you just write the floating-point operation, and the assembler produces the right

opcode depending on which register name you use: S for single and D for double. For example, the ARMv8 assembler accepts

```
FADD D0, D1, D2
```

and produces the double-precision version of the opcode for FADD, which LEGv8 associates with FADDD.

- In the full ARMv8 instruction set, register 31 is XZR in most instructions but the *stack pointer* (SP) in others. We think it is confusing, so register 31 is always XZR and SP is always register 28 in LEGv8. If you stick to using XZR and SP for register names when using the assembler and simulator, it should not be a problem, but this subtle distinction might show up in visualizations of the state of the processor, for example.
- The immediate fields for ANDI, ORRI, and EORI of the full ARMv8 instruction set are not simple 12-bit immediates that we assume in LEGv8. ARMv8 has an algorithm for encoding immediate values as repeating patterns. This means that some small constants (e.g., 1, 2, 3, 4, and 6) are valid, while others (e.g., 0, 5) are not. Thus, the assembler may insert more instructions than you might expect to create simple constants, or fewer if you pick a lucky large constant. The official definition is the bit pattern can be viewed “as a vector of identical elements of size $e = 2, 4, 8, 16, 32$, or 64 bits. Each element contains the same sub-pattern, that is a single run of 1 to $(e - 1)$ nonzero bits from bit 0 followed by zero bits, then rotated by 0 to $(e - 1)$ bits.” Don’t try to figure this out yourself; just leave it to the assembler!

2.1 [5] <§2.2> For the following C statement, write the corresponding LEGv8 assembly code. Assume that the C variables *f*, *g*, and *h*, have already been placed in registers X0, X1, and X2 respectively. Use a minimal number of LEGv8 assembly instructions.

```
f = g + (h - 5);
```

2.2 [5] <§2.2> Write a single C statement that corresponds to the two LEGv8 assembly instructions below.

```
ADD f, g, h
ADD f, i, f
```

2.3 [5] <§§2.2, 2.3> For the following C statement, write the corresponding LEGv8 assembly code. Assume that the variables *f*, *g*, *h*, *i*, and *j* are assigned to registers X0, X1, X2, X3, and X4, respectively. Assume that the base address of the arrays *A* and *B* are in registers X6 and X7, respectively.

```
B[8] = A[i-j];
```

2.4 [10] <§§2.2, 2.3> For the LEGv8 assembly instructions below, what is the corresponding C statement? Assume that the variables *f*, *g*, *h*, *i*, and *j* are assigned to registers *X0*, *X1*, *X2*, *X3*, and *X4*, respectively. Assume that the base address of the arrays *A* and *B* are in registers *X6* and *X7*, respectively.

```

LSL  X9, X0, #3      // X9 = f*8
ADD  X9, X6, X9       // X9 = &A[f]
LSL  X10, X1, #3     // X10 = g*8
ADD  X10, X7, X10    // X10 = &B[g]
LDUR X0, [X9, #0]    // f = A[f]

      ADDI X11, X9, #8
      LDUR X9, [X11, #0]
      ADD  X9, X9, X0
      STUR X9, [X10, #0]
```

2.5 [5] <§§2.2, 2.3, 2.6> For the LEGv8 assembly instructions in Exercise 2.4, rewrite the assembly code to minimize the number of LEGv8 instructions needed to carry out the same function.

2.6 [5] <§2.3> Show how the value *0xabcdef12* would be arranged in memory of a little-endian and a big-endian machine. Assume the data are stored starting at address 0 and that the word size is 4 bytes.

2.7 [5] <§2.4> Translate *0xabcdef12* into decimal.

2.8 [5] <§§2.2, 2.3> Translate the following C code to LEGv8. Assume that the variables *f*, *g*, *h*, *i*, and *j* are assigned to registers *X0*, *X1*, *X2*, *X3*, and *X4*, respectively. Assume that the base address of the arrays *A* and *B* are in registers *X6* and *X7*, respectively. Assume that the elements of the arrays *A* and *B* are 8-byte words:

```
B[8] = A[i] + A[j];
```

2.9 [10] <§§2.2, 2.3> Translate the following LEGv8 code to C. Assume that the variables *f*, *g*, *h*, *i*, and *j* are assigned to registers *X0*, *X1*, *X2*, *X3*, and *X4*, respectively. Assume that the base address of the arrays *A* and *B* are in registers *X6* and *X7*, respectively.

```

      ADDI X9, X6, #8
      ADD  X10, X6, XZR
      STUR X10, [X9, #0]
      LDUR X9, [X9, #0]
      ADD  X0, X9, X10
```

2.10 [20] <§§2.2, 2.5> For each LEGv8 instruction in Exercise 2.9, show the value of the opcode (Op), source register (Rn), and target register (Rd or Rt) fields. For the I-type instructions, show the value of the immediate field, and for the R-type instructions, show the value of the second source register (Rm).

2.11 Assume that registers X0 and X1 hold the values 0×8000000000000000 and 0×D000000000000000, respectively.

2.11.1 [5] <§2.4> What is the value of X9 for the following assembly code?

```
ADD X9, X0, X1
```

2.11.2 [5] <§2.4> Is the result in X9 the desired result, or has there been overflow?

2.11.3 [5] <§2.4> For the contents of registers X0 and X1 as specified above, what is the value of X9 for the following assembly code?

```
SUB X9, X0, X1
```

2.11.4 [5] <§2.4> Is the result in X9 the desired result, or has there been overflow?

2.11.5 [5] <§2.4> For the contents of registers X0 and X1 as specified above, what is the value of X9 for the following assembly code?

```
ADD X9, X0, X1
ADD X9, X9, X0
```

2.11.6 [5] <§2.4> Is the result in X9 the desired result, or has there been overflow?

2.12 Assume that X0 holds the value 128_{ten}.

2.12.1 [5] <§2.4> For the instruction ADD X9, X0, X1, what is the range(s) of values for X1 that would result in overflow?

2.12.2 [5] <§2.4> For the instruction SUB X9, X0, X1, what is the range(s) of values for X1 that would result in overflow?

2.12.3 [5] <§2.4> For the instruction SUB X9, X1, X0, what is the range(s) of values for X1 that would result in overflow?

2.13 [5] <§§2.2, 2.5> Provide the instruction type and assembly language instruction for the following binary value:

```
1000 1011 0000 0000 0000 0000 0000 0000two
```

Hint: Figure 2.20 may be helpful.

2.14 [5] <§§2.2, 2.5> Provide the instruction type and hexadecimal representation of the following instruction:

```
STUR X9, [X10, #32]
```

2.15 [5] <§2.5> Provide the instruction type, assembly language instruction, and binary representation of instruction described by the following LEGv8 fields:

op=0x658, Rm=13, Rn=15, Rd=17, shamt=0

2.16 [5] <§2.5> Provide the instruction type, assembly language instruction, and binary representation of instruction described by the following LEGv8 fields:

op=0x7c2, Rn=12, Rt=3, const=0x4

2.17 Assume that we would like to expand the LEGv8 register file to 128 registers and expand the instruction set to contain four times as many instructions.

2.17.1 [5] <§2.5> How would this affect the size of each of the bit fields in the R-type instructions?

2.17.2 [5] <§2.5> How would this affect the size of each of the bit fields in the I-type instructions?

2.17.3 [5] <§§2.5, 2.8, 2.10> How could each of the two proposed changes decrease the size of a LEGv8 assembly program? On the other hand, how could the proposed change increase the size of an LEGv8 assembly program?

2.18 Assume the following register contents:

X10 = 0x00000000AAAAAAAA, X11 = 0x1234567812345678

2.18.1 [5] <§2.6> For the register values shown above, what is the value of X12 for the following sequence of instructions?

```
LSL X12, X10, #4
ORR X12, X12, X11
```

2.18.2 [5] <§2.6> For the register values shown above, what is the value of X12 for the following sequence of instructions?

```
LSL X12, X11, #4
```

2.18.3 [5] <§2.6> For the register values shown above, what is the value of X12 for the following sequence of instructions?

```
LSR X12, X10, #3
ANDI X12, X12, 0xFEf
```

2.19 [10] <§2.6> Find the shortest sequence of LEGv8 instructions that extracts bits 16 down to 11 from register X10 and uses the value of this field to replace bits 31 down to 26 in register X11 without changing the other bits of registers X10 or X11. (Be sure to test your code using X10 = 0 and X11 = 0xffffffffffffffff. Doing so may reveal a common oversight.)

2.20 [5] <\$2.6> Provide a minimal set of LEGv8 instructions that may be used to implement the following pseudoinstruction:

```
NOT X10, X11          // bit-wise invert
```

2.21 [5] <\$2.6> For the following C statement, write a minimal sequence of LEGv8 assembly instructions that performs the identical operation. Assume X11 = A, and X13 is the base address of C.

```
A = C[0] << 4;
```

2.22 [5] <\$2.7> Assume X0 holds the value 0x0000000000101000. What is the value of X1 after the following instructions?

```
        CMP X0, #0
        B.GE ELSE
        B DONE
ELSE:   ORRI X1, XZR, #2
DONE:
```

2.23 Suppose the *program counter* (PC) is set to 0x2000 0000.

2.23.1 [5] <\$2.10> What range of addresses can be reached using the LEGv8 *branch* (B) instruction? (In other words, what is the set of possible values for the PC after the branch instruction executes?).

2.23.2 [5] <\$2.10> What range of addresses can be reached using the LEGv8 *conditional branch-on-equal* (CBZ) instruction? (In other words, what is the set of possible values for the PC after the branch instruction executes?)

2.24 Consider a proposed new instruction named RPT. This instruction combines a loop's condition check and counter decrement into a single instruction. For example RPT X12, loop would do the following:

```
if (X12 > 0) {
    X12 = X12 - 1;
    goto loop
}
```

2.24.1 [5] <\$2.7, 2.10> If this instruction were to be added to the ARMv8 instruction set, what is the most appropriate instruction format?

2.24.2 [5] <\$2.7> What is the shortest sequence of LEGv8 instructions that performs the same operation?

2.25 Consider the following LEGv8 loop:

```
LOOP:  SUBIS X1, X1, #0
        B.LE DONE
        SUBI X1, X1, #1
        ADDI X0, X0, #2
        B LOOP
DONE:
```

2.25.1 [5] <§2.7> Assume that the register X1 is initialized to the value 10. What is the final value in register X0 assuming the X0 is initially zero?

2.25.2 [5] <§2.7> For the loop above, write the equivalent C code. Assume that the registers X0, and X1 are integers `acc` and `i` respectively.

2.25.3 [5] <§2.7> For the loop written in LEGv8 assembly above, assume that the register X1 is initialized to the value N. How many LEGv8 instructions are executed?

2.25.4 [5] <§2.7> For the loop written in LEGv8 assembly above, replace the instruction “B.LE DONE” instruction with “B.MI DONE”. What is the final value in register X0 assuming the X0 is initially zero?

2.25.5 [5] <§2.7> For the loop written in LEGv8 assembly above, replace the instruction “B.LE DONE” instruction with “B.MI DONE” and write equivalent C code.

2.25.6 [5] <§2.7> What is the purpose of the SUBIS instruction in the assembly code above?

2.25.7 [5] <§2.7> Show how you can reduce the number of instructions by combining the SUBIS and SUBI instructions. (Hint: Add one instruction outside the loop.)

2.26 [10] <§2.7> Translate the following C code to LEGv8 assembly code. Use a minimum number of instructions. Assume that the values of `a`, `b`, `i`, and `j` are in registers X0, X1, X10, and X11, respectively. Also, assume that register X2 holds the base address of the array D.

```
for(i=0; i<a; i++)
    for(j=0; j<b; j++)
        D[4*j] = i + j;
```

2.27 [5] <§2.7> How many LEGv8 instructions does it take to implement the C code from Exercise 2.26? If the variables `a` and `b` are initialized to 10 and 1 and all elements of D are initially 0, what is the total number of LEGv8 instructions executed to complete the loop?

2.28 [5] <§2.7> Translate the following loop into C. Assume that the C-level integer *i* is held in register X10, X0 holds the C-level integer called *result*, and X1 holds the base address of the integer *MemArray*.

```

        ORR X10, XZR, XZR
LOOP:   LDUR X11, [X1, #0]
        ADD X0, X0, X11
        ADDI X1, X1, #8
        ADDI X10, X10, #1
        CMPI X10, 100
        B.LT LOOP

```

2.29 [10] <§2.7> Rewrite the loop from Exercise 2.28 to reduce the number of LEGv8 instructions executed. Hint: Notice that variable *i* is used only for loop control.

2.30 [30] <§2.8> Implement the following C code in LEGv8 assembly. Hint: Remember that the stack pointer must remain aligned on a multiple of 16.

```

int fib(int n){
    if (n==0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fib(n-1) + fib(n-2);
}

```

2.31 [20] <§2.8> For each function call, show the contents of the stack after the function call is made. Assume the stack pointer is originally at address 0x7ffffffc, and follow the register conventions as specified in [Figure 2.12](#).

2.32 [20] <§2.8> Translate function *f* into LEGv8 assembly language. If you need to use registers X10 through X27, use the lower-numbered registers first. Assume the function declaration for *g* is “int *g*(int *a*, int *b*)”. The code for function *f* is as follows:

```

int f(int a, int b, int c, int d){
    return g(g(a,b),c+d);
}

```

2.33 [5] <§2.8> Can we use the tail-call optimization in this function? If no, explain why not. If yes, what is the difference in the number of executed instructions in *f* with and without the optimization?

2.34 [5] <§2.8> Right before your function `f` from Exercise 2.32 returns, what do we know about contents of registers `X5`, `X29`, `X30`, and `SP`? Keep in mind that we know what the entire function `f` looks like, but for function `g` we only know its declaration.

2.35 [30] <§2.9> Write a program in LEGv8 assembly to convert an ASCII string containing a positive or negative integer decimal string to an integer. Your program should expect register `X0` to hold the address of a null-terminated string containing an optional `+` or `-` followed by some combination of the digits 0 through 9. Your program should compute the integer value equivalent to this string of digits, then place the number in register `X0`. If a non-digit character appears anywhere in the string, your program should stop with the value `-1` in register `X0`. For example, if register `X0` points to a sequence of three bytes `50ten`, `52ten`, `0ten` (the null-terminated string “24”), then when the program stops, register `X0` should contain the value `24ten`. The ARMv8 `MUL` instruction takes two registers as input. There is no “`MULI`” instruction. Thus, just store the constant 10 in a register.

2.36 Consider the following code:

```
LDURB X10, [X11, #0]
STUR  X10, [X11, #8]
```

Assume that the register `X11` contains the address `0x10000000` and the data at address is `0x1122334455667788`.

2.36.1 [5] <§2.3, 2.9> What value is stored in `0x10000008` on a big-endian machine?

2.36.2 [5] <§2.3, 2.9> What value is stored in `0x10000008` on a little-endian machine?

2.37 [5] <§2.10> Write the LEGv8 assembly code that creates the 64-bit constant `0x1122334455667788two` and stores that value to register `X0`.

2.38 [10] <§2.11> Write the LEGv8 assembly code to implement the following C code:

```
lock(1k);
shvar=max(shvar,x);
unlock(1k);
```

Assume that the address of the `1k` variable is in `X0`, the address of the `shvar` variable is in `X1`, and the value of variable `x` is in `X2`. Your critical section should not contain any function calls. Use `LDXR`/`STXR` instructions to implement the `lock()` operation, the `unlock()` operation is simply an ordinary store instruction.

2.39 [10] <§2.11> Repeat Exercise 2.38, but this time use `LDXR/STXR` to perform an atomic update of the `shvar` variable directly, without using `lock()` and `unlock()`. Note that in this exercise there is no variable `lk`.

2.40 [5] <§2.11> Using your code from Exercise 2.38 as an example, explain what happens when two processors begin to execute this critical section at the same time, assuming that each processor executes exactly one instruction per cycle.

2.41 Assume for a given processor the CPI of arithmetic instructions is 1, the CPI of load/store instructions is 10, and the CPI of branch instructions is 3. Assume a program has the following instruction breakdowns: 500 million arithmetic instructions, 300 million load/store instructions, 100 million branch instructions.

2.41.1 [5] <§§1.6, 2.13> Suppose that new, more powerful arithmetic instructions are added to the instruction set. On average, through the use of these more powerful arithmetic instructions, we can reduce the number of arithmetic instructions needed to execute a program by 25%, while increasing the clock cycle time by only 10%. Is this a good design choice? Why?

2.41.2 [5] <§§1.6, 2.13> Suppose that we find a way to double the performance of arithmetic instructions. What is the overall speedup of our machine? What if we find a way to improve the performance of arithmetic instructions by 10 times?

2.42 Assume that for a given program 70% of the executed instructions are arithmetic, 10% are load/store, and 20% are branch.

2.42.1 [5] <§§1.6, 2.13> Given this instruction mix and the assumption that an arithmetic instruction requires two cycles, a load/store instruction takes six cycles, and a branch instruction takes three cycles, find the average CPI.

2.42.2 [5] <§§1.6, 2.13> For a 25% improvement in performance, how many cycles, on average, may an arithmetic instruction take if load/store and branch instructions are not improved at all?

2.42.3 [5] <§§1.6, 2.13> For a 50% improvement in performance, how many cycles, on average, may an arithmetic instruction take if load/store and branch instructions are not improved at all?

2.43 [10] <§2.19> Use the full ARMv8 instruction set (in particular the ability to scale a register offset when loading data) to further reduce the number of assembly instructions needed to carry out the function given in Exercise 2.4.

2.44 [10] <§2.19> Use the full ARMv8 instruction set (in particular the ability to scale a register offset when loading data) to further reduce the number of assembly instructions needed to implement the C code given in Exercise 2.8.

2.45 [10] <§2.19> Use the full ARMv8 instruction set to further reduce the number of instructions needed to complete Exercise 2.19.

2.46 [10] <§2.19> Provide a minimal set of ARMv8 instructions that may be used to implement the following pseudoinstruction:

```
NOT X10, X11          // bit-wise invert
```

(Note that some LEGv8 answers to Exercise 2.20 may not be valid ARMv8.)

2.47 [10] <§2.19> Use the pre-indexed addressing mode to minimize the number of ARMv8 assembly instructions needed to implement the code given in Exercise 2.28.

2.48 [5] <§2.19> Use the post-indexed addressing mode to minimize the number of ARMv8 assembly instructions needed to implement the code given in Exercise 2.28.

2.49 [5] <§2.19> Implement an ARMv8 assembly function to find the maximum value in an integer array. Optimize your function to require as few instructions as possible. Hint: using the `CSEL` instruction, it is possible to reduce the body of the loop to five instructions.

Answers to Check Yourself

§2.2, page 67: ARMv8, C, Java.

§2.3, page 73: 2) Very slow.

§2.4, page 81: 2) -8_{ten}

§2.5, page 89: 3) `SUB X11, X10, X9`

§2.6, page 92: Both. `AND` with a mask pattern of 1s will leaves 0s everywhere but the desired field. Shifting left by the correct amount removes the bits from the left of the field. Shifting right by the appropriate amount puts the field into the rightmost bits of the doubleword, with 0s in the rest of the doubleword. Note that `AND` leaves the field where it was originally, and the shift pair moves the field into the rightmost part of the doubleword.

§2.7, page 99: I. All are true. II. 1).

§2.8, page 110: Both are true.

§2.9, page 115: I. 1) and 2) II. 3).

§2.10, page 124: I. 4) ± 1024 K. II. 4) ± 128 M.

§2.11, page 128: Both are true.

§2.12, page 137: 4) Machine independence.

This page intentionally left blank

3

*Numerical precision
is the very soul of
science.*

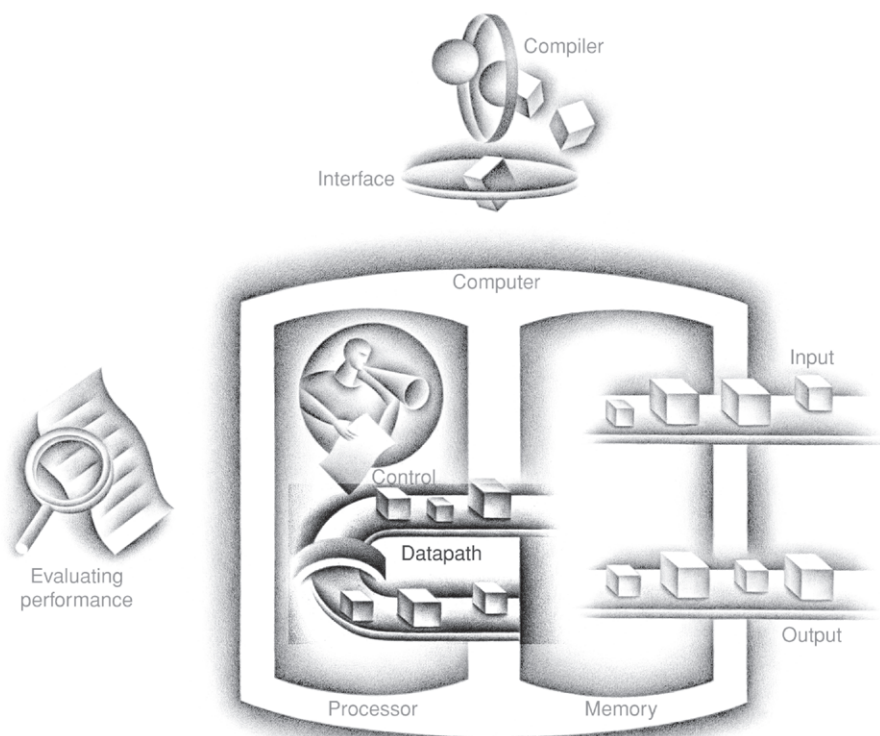
Sir D'arcy Wentworth Thompson,
On Growth and Form, 1917

Arithmetic for Computers

- 3.1 Introduction** 188
- 3.2 Addition and Subtraction** 188
- 3.3 Multiplication** 191
- 3.4 Division** 197
- 3.5 Floating Point** 205
- 3.6 Parallelism and Computer Arithmetic:
Subword Parallelism** 230
- 3.7 Real Stuff: Streaming SIMD Extensions and
Advanced Vector Extensions in x86** 232

3.8	Real Stuff: The Rest of the ARMv8 Arithmetic Instructions	234
3.9	Going Faster: Subword Parallelism and Matrix Multiply	238
3.10	Fallacies and Pitfalls	242
3.11	Concluding Remarks	245
 3.12	Historical Perspective and Further Reading	248
3.13	Exercises	249

The Five Classic Components of a Computer



3.1 Introduction

Computer words are composed of bits; thus, words can be represented as binary numbers. Chapter 2 shows that integers can be represented either in decimal or binary form, but what about the other numbers that commonly occur? For example:

- What about fractions and other real numbers?
- What happens if an operation creates a number bigger than can be represented?
- And underlying these questions is a mystery: How does hardware really multiply or divide numbers?

The goal of this chapter is to unravel these mysteries—including representation of real numbers, arithmetic algorithms, hardware that follows these algorithms—and the implications of all this for instruction sets. These insights may explain quirks that you have already encountered with computers. Moreover, we show how to use this knowledge to make arithmetic-intensive programs go much faster.

3.2 Addition and Subtraction

Addition is just what you would expect in computers. Digits are added bit by bit from right to left, with carries passed to the next digit to the left, just as you would do by hand. Subtraction uses addition: the appropriate operand is simply negated before being added.

Subtraction: Addition's Tricky Pal
No. 10, Top Ten Courses for Athletes at a Football Factory, David Letterman et al., *Book of Top Ten Lists*, 1990

EXAMPLE

Binary Addition and Subtraction

Let's try adding 6_{ten} to 7_{ten} in binary and then subtracting 6_{ten} from 7_{ten} in binary.

$$\begin{array}{r} 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000111_{two} = 7_{ten} \\ +\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000110_{two} = 6_{ten} \\ \hline =\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00001101_{two} = 13_{ten} \end{array}$$

The 4 bits to the right have all the action; Figure 3.1 shows the sums and carries. Parentheses identify the carries, with the arrows illustrating how they are passed.

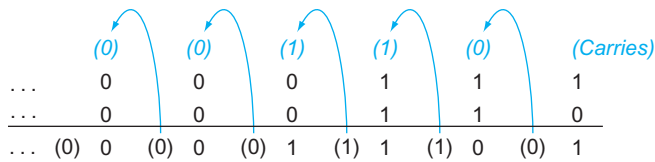


FIGURE 3.1 Binary addition, showing carries from right to left. The rightmost bit adds 1 to 0, resulting in the sum of this bit being 1 and the carry out from this bit being 0. Hence, the operation for the second digit to the right is $0 + 1 + 1$. This generates a 0 for this sum bit and a carry out of 1. The third digit is the sum of $1 + 1 + 1$, resulting in a carry out of 1 and a sum bit of 1. The fourth bit is $1 + 0 + 0$, yielding a 1 sum and no carry.

Subtracting 6_{ten} from 7_{ten} can be done directly:

$$\begin{array}{r}
 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000111_{\text{two}} = 7_{\text{ten}} \\
 +\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000110_{\text{two}} = 6_{\text{ten}} \\
 \hline
 =\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000001_{\text{two}} = 1_{\text{ten}}
 \end{array}$$

or via addition using the two's complement representation of -6 :

$$\begin{array}{r}
 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000111_{\text{two}} = 7_{\text{ten}} \\
 +\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111010_{\text{two}} = -6_{\text{ten}} \\
 \hline
 =\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000000\ 00000001_{\text{two}} = 1_{\text{ten}}
 \end{array}$$

Recall that overflow occurs when the result from an operation cannot be represented with the available hardware, in this case a 64-bit word. When can overflow occur in addition? When adding operands with different signs, overflow cannot occur. The reason is the sum must be no larger than one of the operands. For example, $-10 + 4 = -6$. Since the operands fit in 64 bits and the sum is no larger than an operand, the sum must fit in 64 bits as well. Therefore, no overflow can occur when adding positive and negative operands.

There are similar restrictions to the occurrence of overflow during subtract, but it's just the opposite principle: when the signs of the operands are the *same*, overflow cannot occur. To see this, remember that $c - a = c + (-a)$ because we subtract by negating the second operand and then add. Therefore, when we subtract operands of the same sign we end up *adding* operands of *different* signs. From the prior paragraph, we know that overflow cannot occur in this case either.

Knowing when an overflow cannot occur in addition and subtraction is all well and good, but how do we detect it when it *does* occur? Clearly, adding or subtracting two 64-bit numbers can yield a result that needs 65 bits to be fully expressed.

ANSWER

Operation	Operand A	Operand B	Result indicating overflow
$A + B$	≥ 0	≥ 0	< 0
$A + B$	< 0	< 0	≥ 0
$A - B$	≥ 0	< 0	< 0
$A - B$	< 0	≥ 0	≥ 0

FIGURE 3.2 Overflow conditions for addition and subtraction.

The lack of a 65th bit means that when an overflow occurs, the sign bit is set with the *value* of the result instead of the proper sign of the result. Since we need just one extra bit, only the sign bit can be wrong. Hence, overflow occurs when adding two positive numbers and the sum is negative, or vice versa. This spurious sum means a carry out occurred into the sign bit.

Overflow occurs in subtraction when we subtract a negative number from a positive number and get a negative result, or when we subtract a positive number from a negative number and get a positive result. Such a ridiculous result means a borrow occurred from the sign bit. Figure 3.2 shows the combination of operations, operands, and results that indicate an overflow.

We have just seen how to detect overflow for two’s complement numbers in a computer. What about overflow with unsigned integers? Unsigned integers are commonly used for memory addresses where overflows are ignored.

Given that overflow is one of the four condition codes, it is easy for the compiler to check for overflow for add and subtract instructions if it needs to. The programming language C ignores overflows, but the programming language Fortran needs them discovered.

Appendix A describes the hardware that performs addition and subtraction, which is called an Arithmetic Logic Unit or ALU.

Arithmetic Logic Unit (ALU) Hardware that performs addition, subtraction, and usually logical operations such as AND and OR.

Hardware/
Software
Interface

The computer designer must decide how to handle arithmetic overflows. Although some languages like C and Java ignore integer overflow, languages like Ada and Fortran require that the program be notified. The programmer or the programming environment must then decide what to do when an overflow occurs.

Summary

A major point of this section is that, independent of the representation, the finite word size of computers means that arithmetic operations can create results that are too large to fit in this fixed word size. It’s easy to detect overflow in unsigned numbers, although these are almost always ignored because programs don’t want to detect overflow for address arithmetic, the most common use of natural numbers. Two’s complement presents a greater challenge, yet some software systems require recognizing overflow, so today all computers have a way to detect it.

Some programming languages allow two's complement integer arithmetic on variables declared byte and half, whereas the LEGv8 core only has integer arithmetic operations on full words. As we recall from [Chapter 2](#), LEGv8 core does have data transfer operations for bytes and halfwords. What LEGv8 instructions should be generated for byte and halfword arithmetic operations?

Check Yourself

1. Load with LDURB, LDURH; arithmetic with ADD, SUB, MUL, DIV, using AND to mask result to 8 or 16 bits after each operation; then store using STURB, STURH.
2. Load with LDURB, LDURH; arithmetic with ADD, SUB, MUL, DIV; then store using STURB, STURH.

Elaboration: One feature not generally found in general-purpose microprocessors is saturating operations. *Saturation* means that when a calculation overflows, the result is set to the largest positive number or the most negative number, rather than a modulo calculation as in two's complement arithmetic. Saturation is likely what you want for media operations. For example, the volume knob on a radio set would be frustrating if, as you turned it, the volume would get continuously louder for a while and then immediately very soft. A knob with saturation would stop at the highest volume no matter how far you turned it. Multimedia extensions to standard instruction sets often offer saturating arithmetic.

Elaboration: Determining the carry into the high-order bits earlier increases the speed of addition. There are a variety of schemes to anticipate the carry so that the worst-case scenario is a function of the $\log_2$ of the number of bits in the adder. These anticipatory signals are faster because they go through fewer gates in sequence, but it takes many more gates to anticipate the proper carry. The most popular is *carry lookahead*, which [Section A.6 in Appendix A](#) describes.

3.3 Multiplication

Now that we have completed the explanation of addition and subtraction, we are ready to build the more vexing operation of multiplication.

First, let's review the multiplication of decimal numbers in longhand to remind ourselves of the steps of multiplication and the names of the operands. For reasons that will become clear shortly, we limit this decimal example to using only the digits 0 and 1. Multiplying 1000_{ten} by 1001_{ten} :

$$\begin{array}{r}
 \text{Multiplicand} \quad 1000_{\text{ten}} \\
 \text{Multiplier} \quad \times \quad 1001_{\text{ten}} \\
 \hline
 1000 \\
 0000 \\
 0000 \\
 1000 \\
 \hline
 \text{Product} \quad 1001000_{\text{ten}}
 \end{array}$$

Multiplication is vexation, Division is as bad; The rule of three doth puzzle me, And practice drives me mad.

Anonymous,
Elizabethan manuscript,
1570

The first operand is called the *multiplicand* and the second the *multiplier*. The final result is called the *product*. As you may recall, the algorithm learned in grammar school is to take the digits of the multiplier one at a time from right to left, multiplying the multiplicand by the single digit of the multiplier, and shifting the intermediate product one digit to the left of the earlier intermediate products.

The first observation is that the number of digits in the product is considerably larger than the number in either the multiplicand or the multiplier. In fact, if we ignore the sign bits, the length of the multiplication of an n -bit multiplicand and an m -bit multiplier is a product that is $n + m$ bits long. That is, $n + m$ bits are required to represent all possible products. Hence, like add, multiply must cope with overflow because we frequently want a 64-bit product as the result of multiplying two 64-bit numbers.

In this example, we restricted the decimal digits to 0 and 1. With only two choices, each step of the multiplication is simple:

1. Just place a copy of the multiplicand ($1 \times$ multiplicand) in the proper place if the multiplier digit is a 1, or
2. Place 0 ($0 \times$ multiplicand) in the proper place if the digit is 0.

Although the decimal example above happens to use only 0 and 1, multiplication of binary numbers must always use 0 and 1, and thus always offers only these two choices.

Now that we have reviewed the basics of multiplication, the traditional next step is to provide the highly optimized multiply hardware. We break with tradition in the belief that you will gain a better understanding by seeing the evolution of the multiply hardware and algorithm through multiple generations. For now, let's assume that we are multiplying only positive numbers.

Sequential Version of the Multiplication Algorithm and Hardware

This design mimics the algorithm we learned in grammar school; [Figure 3.3](#) shows the hardware. We have drawn the hardware so that data flow from top to bottom to resemble more closely the paper-and-pencil method.

Let's assume that the multiplier is in the 64-bit Multiplier register and that the 128-bit Product register is initialized to 0. From the paper-and-pencil example above, it's clear that we will need to move the multiplicand left one digit each step, as it may be added to the intermediate products. Over 64 steps, a 64-bit multiplicand would move 64 bits to the left. Hence, we need a 128-bit Multiplicand register, initialized with the 64-bit multiplicand in the right half and zero in the left half. This register is then shifted left 1 bit each step to align the multiplicand with the sum being accumulated in the 128-bit Product register.

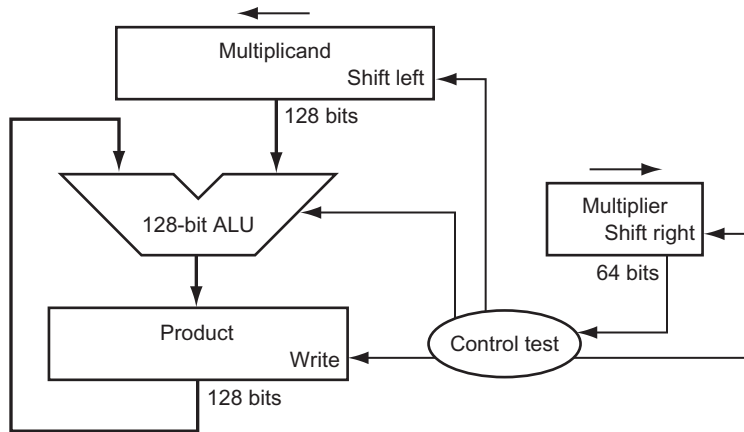


FIGURE 3.3 First version of the multiplication hardware. The Multiplicand register, ALU, and Product register are all 128 bits wide, with only the Multiplier register containing 64 bits. (Appendix A describes ALUs.) The 64-bit multiplicand starts in the right half of the Multiplicand register and is shifted left 1 bit on each step. The multiplier is shifted in the opposite direction at each step. The algorithm starts with the product initialized to 0. Control decides when to shift the Multiplicand and Multiplier registers and when to write new values into the Product register.

Figure 3.4 shows the three basic steps needed for each bit. The least significant bit of the multiplier (Multiplier0) determines whether the multiplicand is added to the Product register. The left shift in step 2 has the effect of moving the intermediate operands to the left, just as when multiplying with paper and pencil. The shift right in step 3 gives us the next bit of the multiplier to examine in the following iteration. These three steps are repeated 64 times to obtain the product. If each step took a clock cycle, this algorithm would require almost 200 clock cycles to multiply two 64-bit numbers. The relative importance of arithmetic operations like multiply varies with the program, but addition and subtraction may be anywhere from 5 to 100 times more popular than multiply. Accordingly, in many applications, multiply can take several clock cycles without significantly affecting performance. However, Amdahl's Law (see Section 1.10) reminds us that even a moderate frequency for a slow operation can limit performance.

This algorithm and hardware are easily refined to take one clock cycle per step. The speed up comes from performing the operations in parallel: the multiplier and multiplicand are shifted while the multiplicand is added to the product if the multiplier bit is a 1. The hardware just has to ensure that it tests the right bit of the multiplier and gets the preshifted version of the multiplicand. The hardware is usually further optimized to halve the width of the adder and registers by noticing where there are unused portions of registers and adders. Figure 3.5 shows the revised hardware.

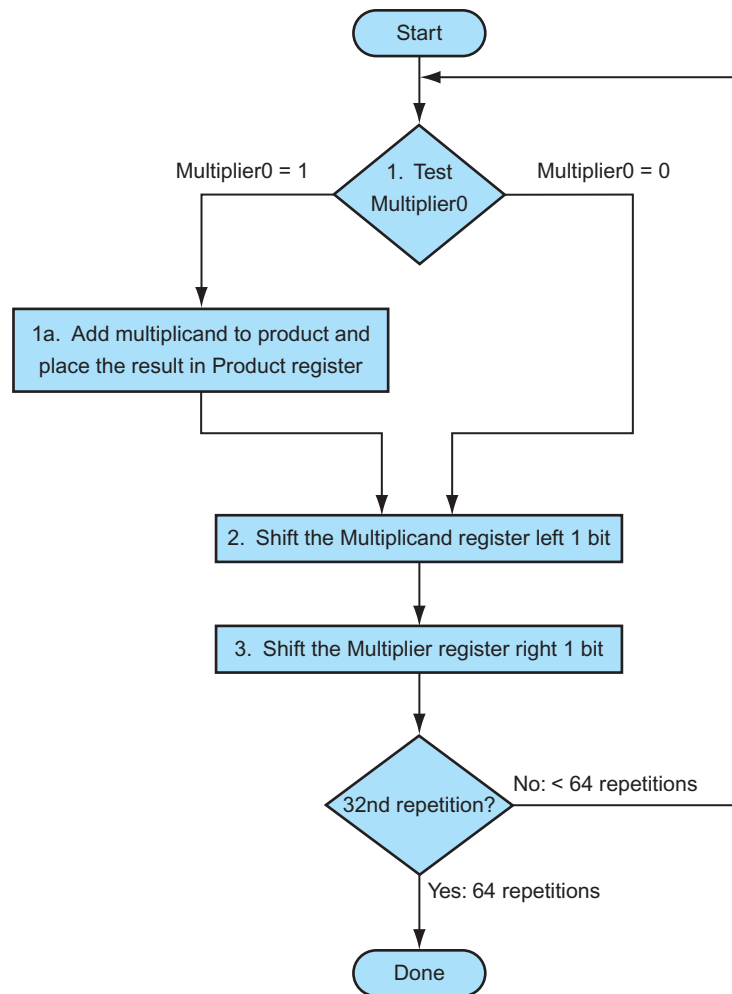


FIGURE 3.4 The first multiplication algorithm, using the hardware shown in Figure 3.3. If the least significant bit of the multiplier is 1, add the multiplicand to the product. If not, go to the next step. Shift the multiplicand left and the multiplier right in the next two steps. These three steps are repeated 64 times.

Hardware/ Software Interface

Replacing arithmetic by shifts can also occur when multiplying by constants. Some compilers replace multiplies by short constants with a series of shifts and adds. Because one bit to the left represents a number twice as large in base 2, shifting the bits left has the same effect as multiplying by a power of 2. As mentioned in Chapter 2, almost every compiler will perform the strength reduction optimization of substituting a left shift for a multiply by a power of 2.

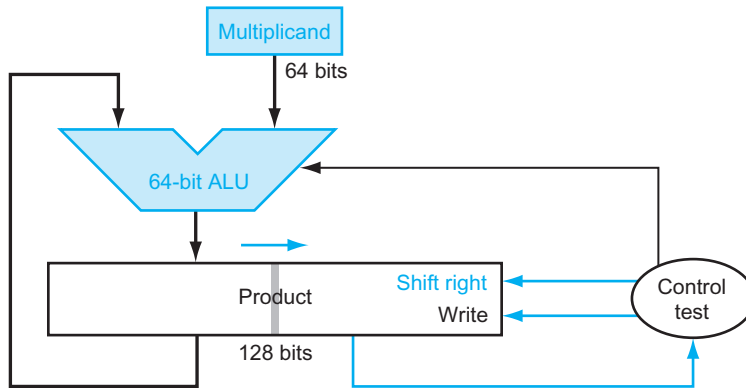


FIGURE 3.5 Refined version of the multiplication hardware. Compare with the first version in Figure 3.3. The Multiplicand register, ALU, and Multiplier register are all 64 bits wide, with only the Product register left at 128 bits. Now the product is shifted right. The separate Multiplier register also disappeared. The multiplier is placed instead in the right half of the Product register. These changes are highlighted in color. (The Product register should really be 129 bits to hold the carry out of the adder, but it's shown here as 128 bits to highlight the evolution from Figure 3.3.)

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	0011	0000 0010	0000 0000
1	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	0001	0000 0100	0000 0010
2	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	0000	0000 1000	0000 0110
3	1: $0 \Rightarrow$ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	0000	0001 0000	0000 0110
4	1: $0 \Rightarrow$ No operation	0000	0001 0000	0000 0110
	2: Shift left Multiplicand	0000	0010 0000	0000 0110
	3: Shift right Multiplier	0000	0010 0000	0000 0110

FIGURE 3.6 Multiply example using algorithm in Figure 3.4. The bit examined to determine the next step is circled in color.

A Multiply Algorithm

Using 4-bit numbers to save space, multiply $2_{\text{ten}} \times 3_{\text{ten}}$, or $0010_{\text{two}} \times 0011_{\text{two}}$.

Figure 3.6 shows the value of each register for each of the steps labeled according to Figure 3.4, with the final value of $0000\ 0110_{\text{two}}$ or 6_{ten} . Color is used to indicate the register values that change on that step, and the bit circled is the one examined to determine the operation of the next step.

EXAMPLE

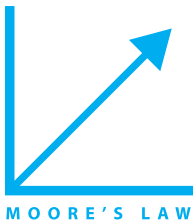
ANSWER

Signed Multiplication

So far, we have dealt with positive numbers. The easiest way to understand how to deal with signed numbers is to first convert the multiplier and multiplicand to positive numbers and then remember their original signs. The algorithms should next be run for 31 iterations, leaving the signs out of the calculation. As we learned in grammar school, we need negate the product only if the original signs disagree.

It turns out that the last algorithm will work for signed numbers, if we remember that we are dealing with numbers that have infinite digits, and we are only representing them with 64 bits. Hence, the shifting steps would need to extend the sign of the product for signed numbers. When the algorithm completes, the lower doubleword would have the 64-bit product.

Faster Multiplication



Moore’s Law has provided so much more in resources that hardware designers can now build much faster multiplication hardware. Whether the multiplicand is to be added or not is known at the beginning of the multiplication by looking at each of the 64 multiplier bits. Faster multiplications are possible by essentially providing one 64-bit adder for each bit of the multiplier: one input is the multiplicand ANDed with a multiplier bit, and the other is the output of a prior adder.

A straightforward approach would be to connect the outputs of adders on the right to the inputs of adders on the left, making a stack of adders 64 high. An alternative way to organize these 64 additions is in a parallel tree, as [Figure 3.7](#) shows. Instead of waiting for 64 add times, we wait just the $\log_2(64)$ or six 64-bit add times.

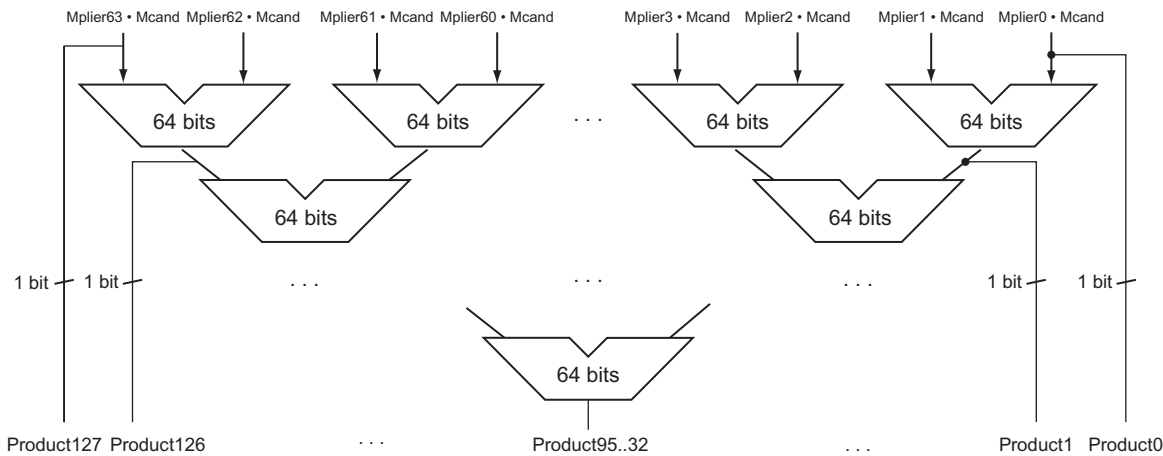


FIGURE 3.7 Fast multiplication hardware. Rather than use a single 64-bit adder 63 times, this hardware “unrolls the loop” to use 63 adders and then organizes them to minimize delay.